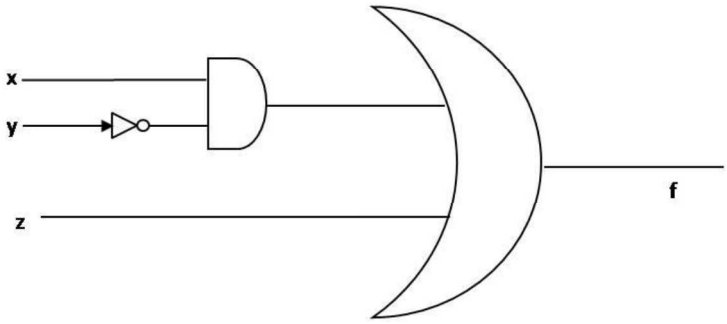
Laborator 3: Implementarea funcțiilor logice folosind porți logice si circuite MSI (multiplexoare, decodificatoare)

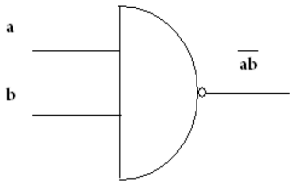
Implementarea funcțiilor logice folosind porți logice

Fie funcția $f = x * \overline{y} + z$. Circuitul corespunzător va fi următorul:

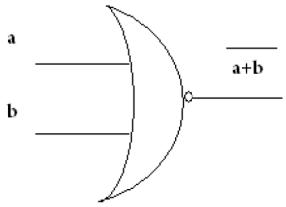


Pentru optimizare, însă, se obișnuiește să se folosească un singur tip de poarta:

- NAND:



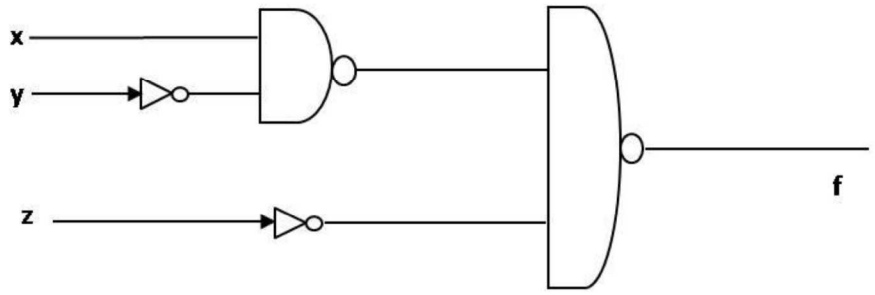
- sau NOR:



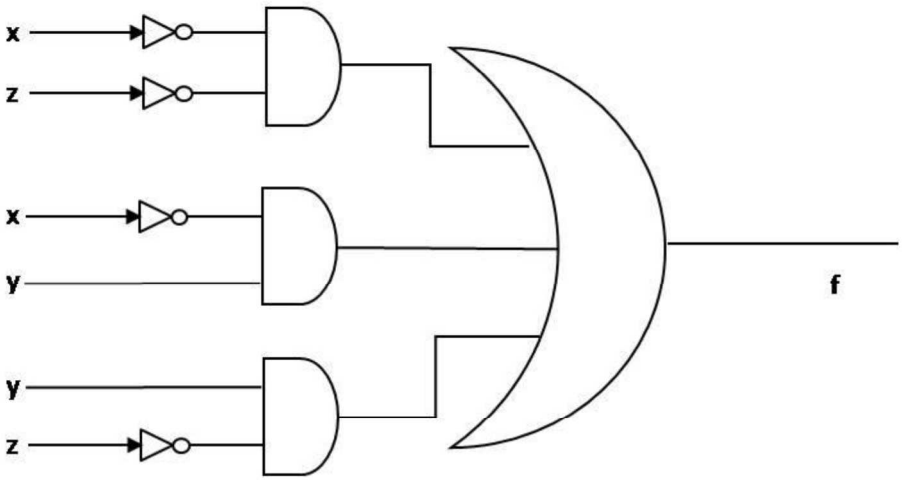
Pentru a ajunge deci sa folosim același tip de poarta in realizarea circuitului lui f, negam de doua ori expresia lui f si aplicam regulile DeMorgan:

$$f = \overline{\overline{f}} = \overline{\overline{x * \overline{y} + z}} = \overline{\overline{x * \overline{y}} * \overline{z}}$$

Circuitul devine:

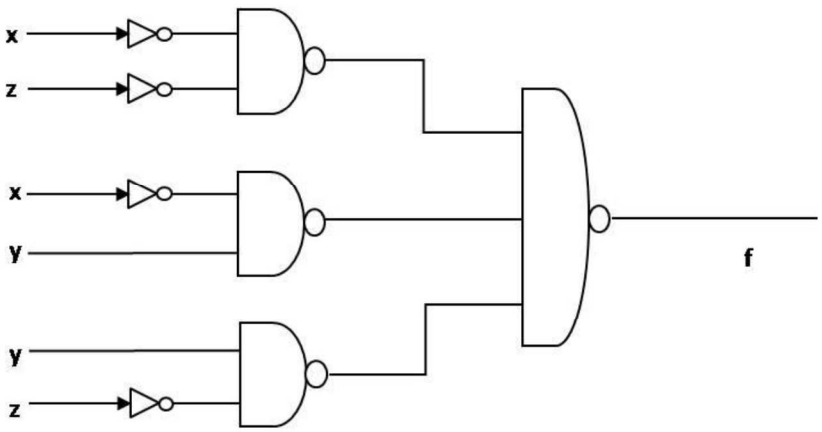


Exemplul 1: Realizați circuitul echivalent al funcției $f = \bar{x} * \bar{z} + \bar{x} * y + y * \bar{z}$:



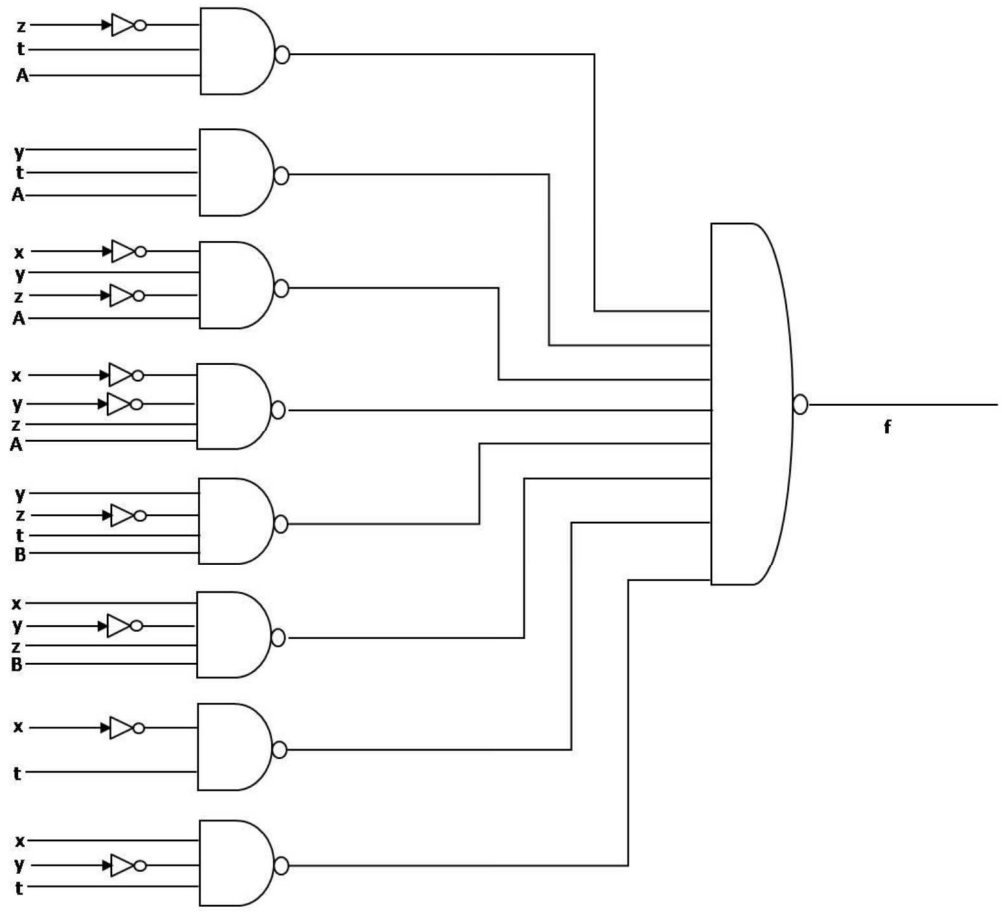
Aplicând relațiile lui deMorgan,se obține funcția :

$$f = \overline{\overline{\bar{x} * \bar{z}} + \overline{\bar{x} * y} + \overline{y * \bar{z}}} = \overline{\overline{\bar{x} * \bar{z}} * \overline{\bar{x} * y} * \overline{y * \bar{z}}}$$



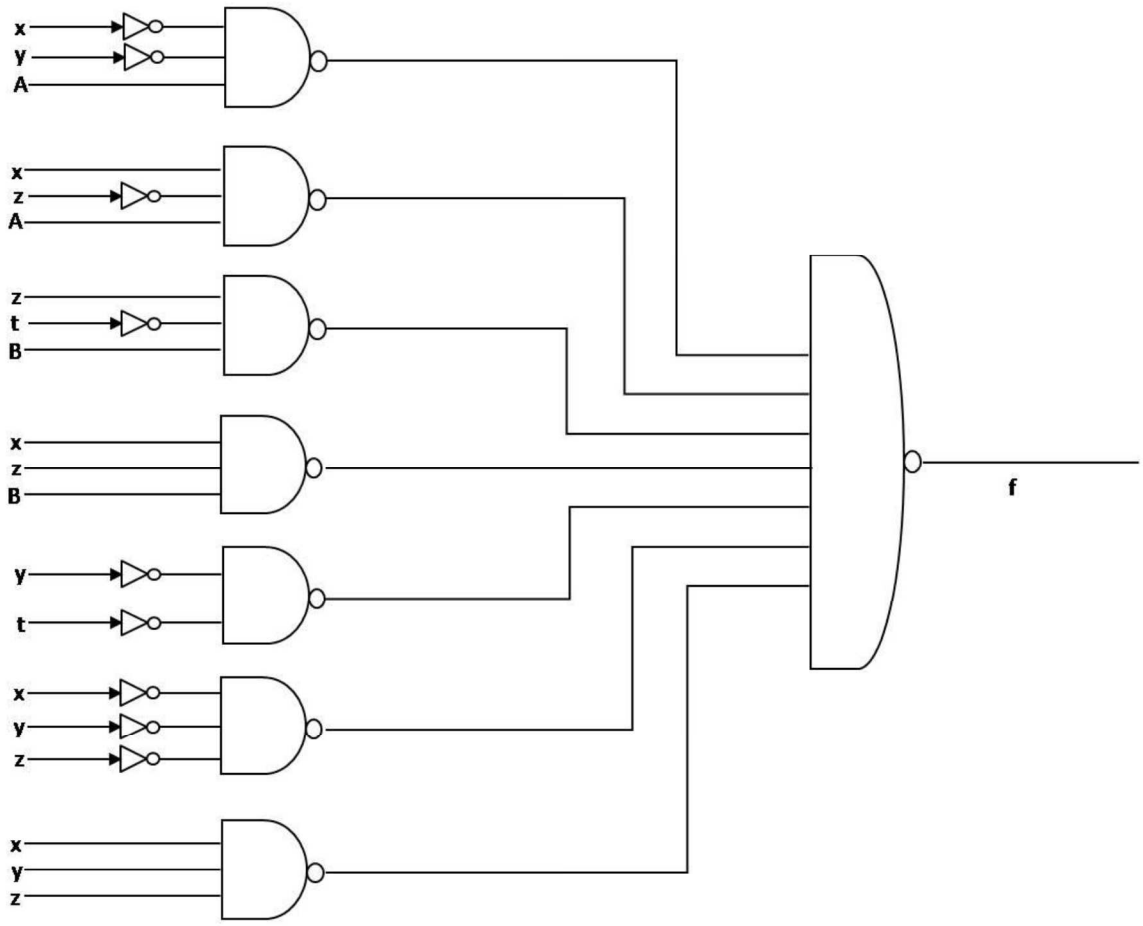
Exemplul 2: Implementați funcția de mai jos folosind circuite de tip NAND:

$$f = \bar{x} * \bar{y} * A + x * \bar{z} * A + z * \bar{t} * B + x * z \quad B + \bar{y} * \bar{t} + \bar{x} * \bar{y} * \bar{z} + x * y * z$$



Exemplul 3: Implementați funcția de mai jos folosind circuite de tip NAND:

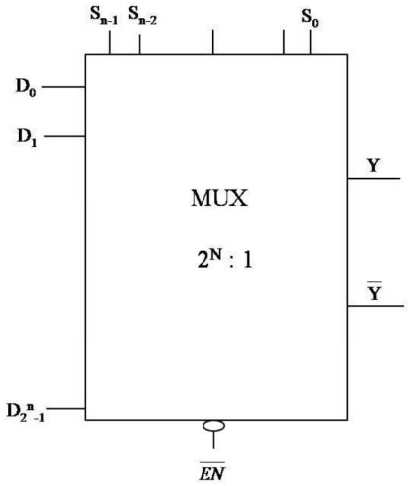
$$f = \overline{x} * \overline{y} * A + x * \overline{z} * A + z * \overline{t} * B \quad x * z * B + \overline{y} * \overline{t} + \overline{x} * \overline{y} * \overline{z} + x * y * z$$



Implementarea funcțiilor logice folosind circuite MSI

Multiplexoare (MUX)

Sunt circuite combinaţionale cu n intrări de selecţie, 2^n intrări de date, o ieşire şi eventual ieşirea negată, şi o intrare de enable. La circuitele combinaţionale, ieşirea la un moment dat depinde doar de intrările de la momentul respectiv.



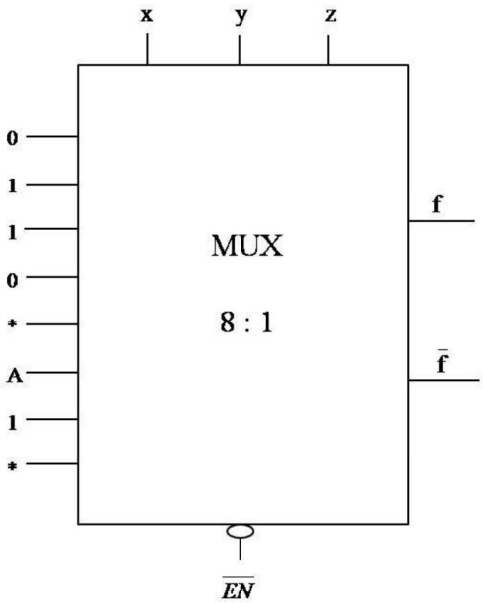
Ieşirea $Y = D_k$, unde k este numărul in baza 10 obţinut din intrările respective.

Observaţie: Pentru a se implementa o funcţie cu multiplexoare, acea funcţie nu se mai minimizează in prealabil!

Exemplul 1: Implementați funcţia de mai jos utilizând MUX 8:1:

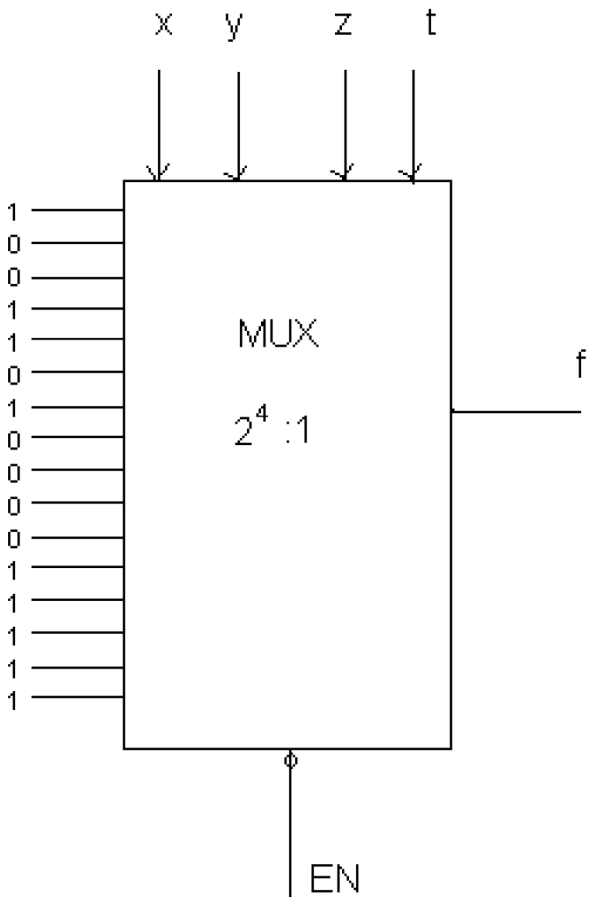
x	y	z	f
0	0	0	**0**
0	0	1	**1**
0	1	0	**1**
0	1	1	**0**
1	0	0	*
1	0	1	**A**
1	1	0	**1**
1	1	1	*

Implementarea funcţiei utilizând MUX 8:1:



Exemplul 2: Sa se implementeze funcția de mai jos folosind un multiplexor 16:1.

x	y	z	t	**f**
0	0	0	0	**1**
0	0	0	1	**0**
0	0	1	0	**0**
0	0	1	1	**1**
0	1	0	0	**1**
0	1	0	1	**0**
0	1	1	0	**1**
0	1	1	1	**0**
1	0	0	0	**0**
1	0	0	1	**0**
1	0	1	0	**0**
1	0	1	1	**1**
1	1	0	0	**1**
1	1	0	1	**1**
1	1	1	0	**1**
1	1	1	1	**1**

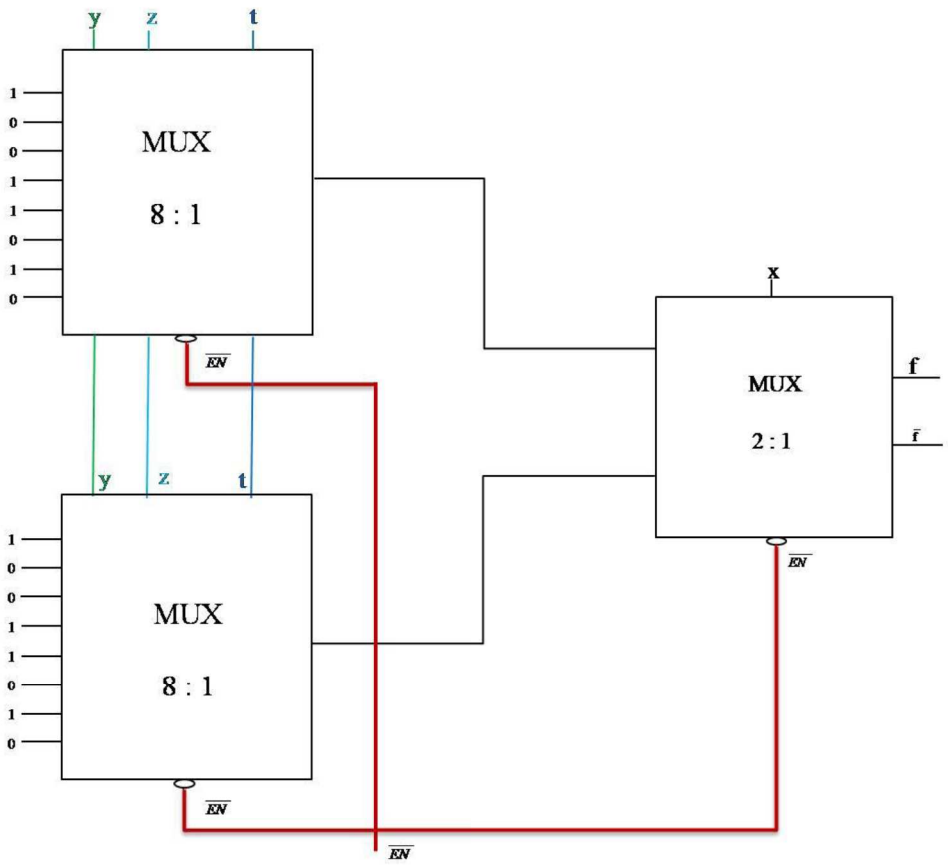


Legarea MUX în cascadă

Practic se descompune un MUX mai mare în mai multe MUX-uri mai mici. Numărul total de intrări în primul nivel al MUX-urilor legate în cascadă este egal cu numărul de intrări din MUX-ul mare şi cu numărul de linii din tabelul de adevăr. Dimensiunea MUX-ului impune numărul de intrări în MUX, dar şi numărul de variabile de selecţie. De aceea, având MUX-uri mai mici, înseamnă că avem mai puţine variabile de selecţie.
Drept urmare, pentru rezolvare, se aleg MUX-urile cerute în problemă şi se fac legăturile, apoi se selectează variabilele de selecţie corespunzătoare fiecărui nivel în parte, pentru ca în final să se identifice valorile corespunzătoare fiecărei combinaţii în parte.

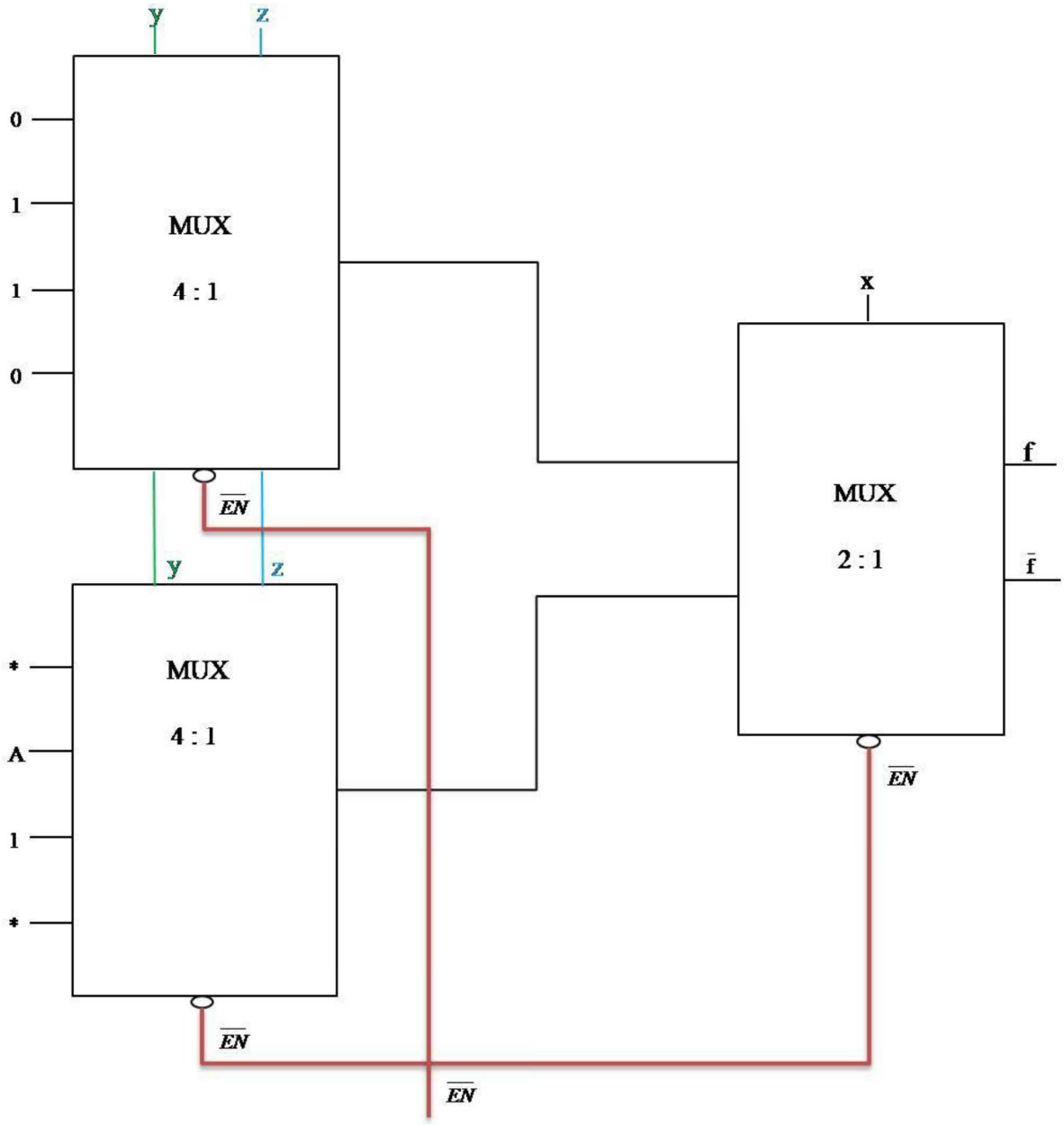
Exemplul 3: Să se implementeze funcţia de mai jos (funcţia din exemplul 2) folosind multiplexoare 8:1:

x	y	z	t	**f**
0	0	0	0	**1**
0	0	0	1	**0**
0	0	1	0	**0**
0	0	1	1	**1**
0	1	0	0	**1**
0	1	0	1	**0**
0	1	1	0	**1**
0	1	1	1	**0**
1	0	0	0	**0**
1	0	0	1	**0**
1	0	1	0	**0**
1	0	1	1	**1**
1	1	0	0	**1**
1	1	0	1	**1**
1	1	1	0	**1**
1	1	1	1	**1**

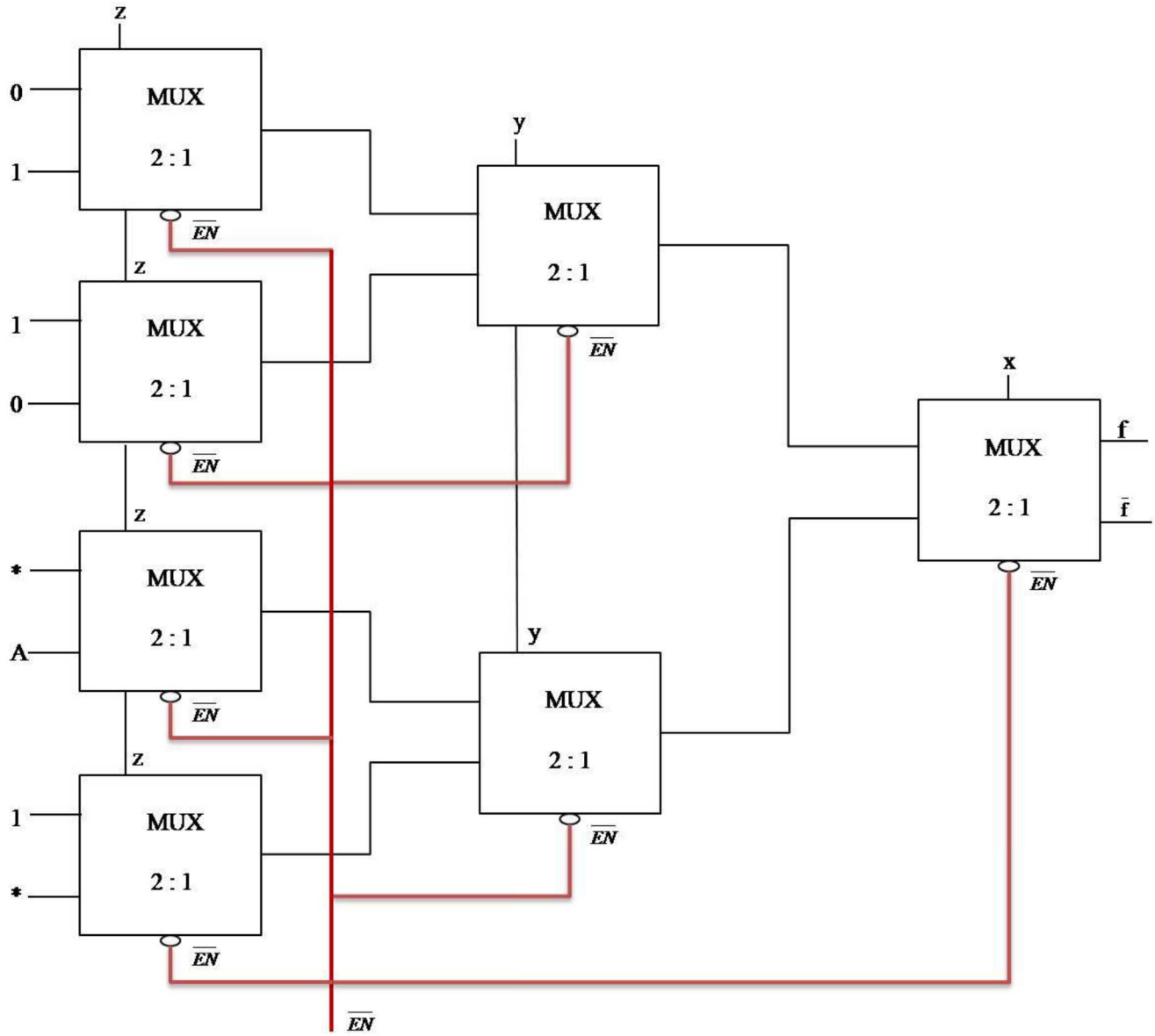


Exemplul 4: Implementați funcția de mai jos (aceeași din exemplul 1) utilizând 2 MUX 4:1 şi un MUX 2:1:

x	y	z	f
0	0	0	**0**
0	0	1	**1**
0	1	0	**1**
0	1	1	**0**
1	0	0	*
1	0	1	**A**
1	1	0	**1**
1	1	1	*


y
z
0
1
1
0
MUX
4 : 1
EN
x
MUX
2 : 1
f
f̄
EN
y
z
*
A
1
*
MUX
4 : 1
EN
EN

Exemplul 5: Implementați aceiași funcție utilizând numai MUX 2:1 (în total vor fi 7 MUX 2:1):

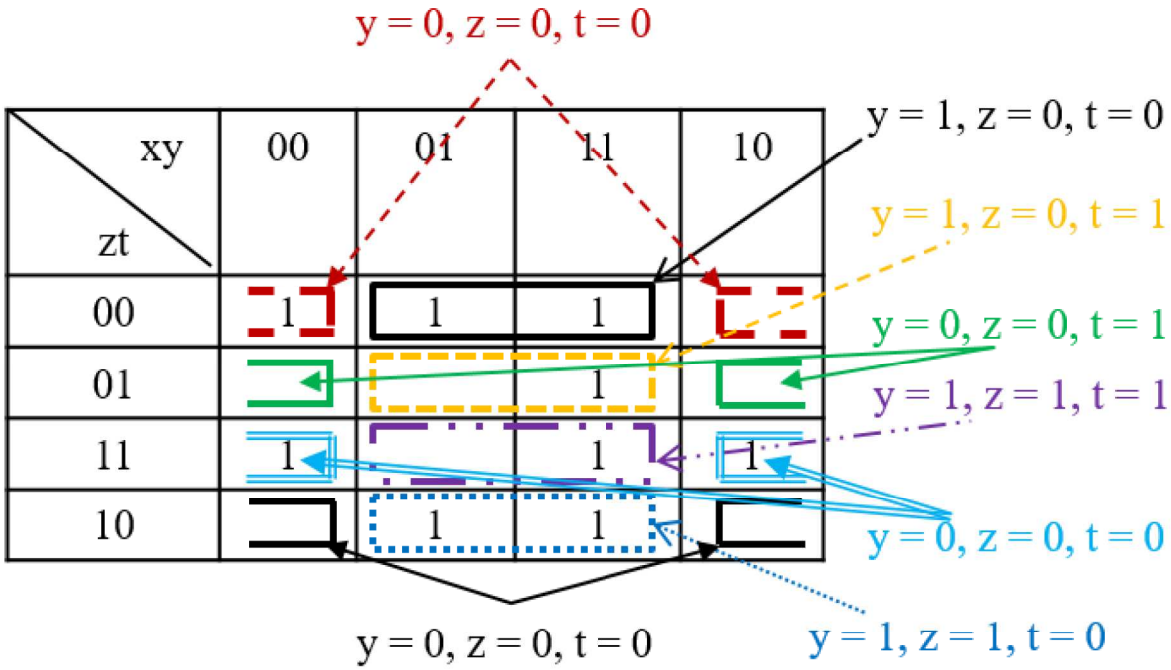


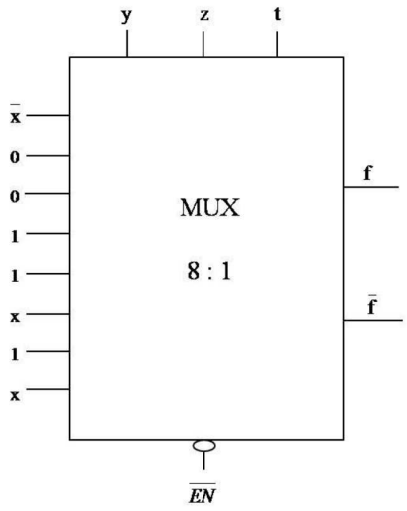
Multiplexarea intrărilor

Această metodă permite reprezentarea unor funcții mai mari prin MUX-uri mai mici. Se bazează pe ideea funcțiilor cu variabile în coloana rezultat. Practic încercăm să restrângem numărul liniilor din tabel prin trecerea uneia sau mai multor variabile în coloana rezultat. Pentru a face acest lucru, cel mai simplu este să se plece de la diagramele Karnaugh, să se identifice zonele corespunzătoare fiecărei intrări din MUX și să se minimizeze zonele respective, conform regulilor învățate în Laboratorul 2.

Exemplul 6: Să se implementeze funcția din exemplul 2 de mai sus folosind un singur multiplexor 8:1 (punând o variabilă în coloana rezultat) și folosind variabilele y, z și t ca variabile de selecție.

x	y	z	t	**f**
0	0	0	0	**1**
0	0	0	1	**0**
0	0	1	0	**0**
0	0	1	1	**1**
0	1	0	0	**1**
0	1	0	1	**0**
0	1	1	0	**1**
0	1	1	1	**0**
1	0	0	0	**0**
1	0	0	1	**0**
1	0	1	0	**0**
1	0	1	1	**1**
1	1	0	0	**1**
1	1	0	1	**1**
1	1	1	0	**1**
1	1	1	1	**1**





Exemplul 7: Să se implementeze funcția de mai sus folosind un singur multiplexor 4:1 (punând două variabile în rezultat):

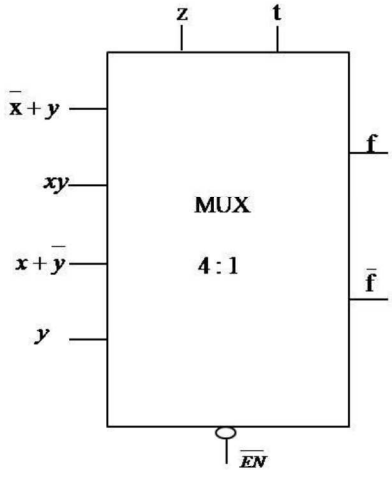
xy / zt	00	01	11	10
00	1	1	1	
01			1	
11	1		1	1
10		1	1	

xy	00	01	11	10
	1	1	1	

xy	00	01	11	10
			1	

	1		1	1
xλ	00	01	11	10

		1	1	
xλ	00	01	11	10

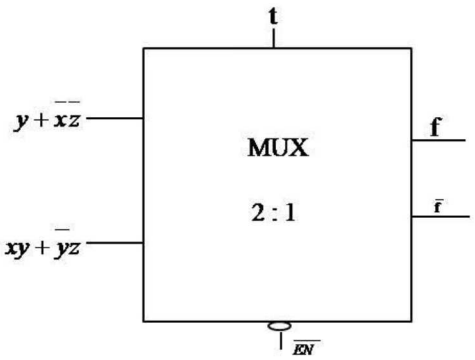


Exemplul 8: Să se implementeze funcția de mai sus folosind un singur multiplexor 2:1 (punând trei variabile în rezultat):

zt \ xy	00	01	11	10
00	1	1	1	
01			1	
11	1		1	1
10		1	1	

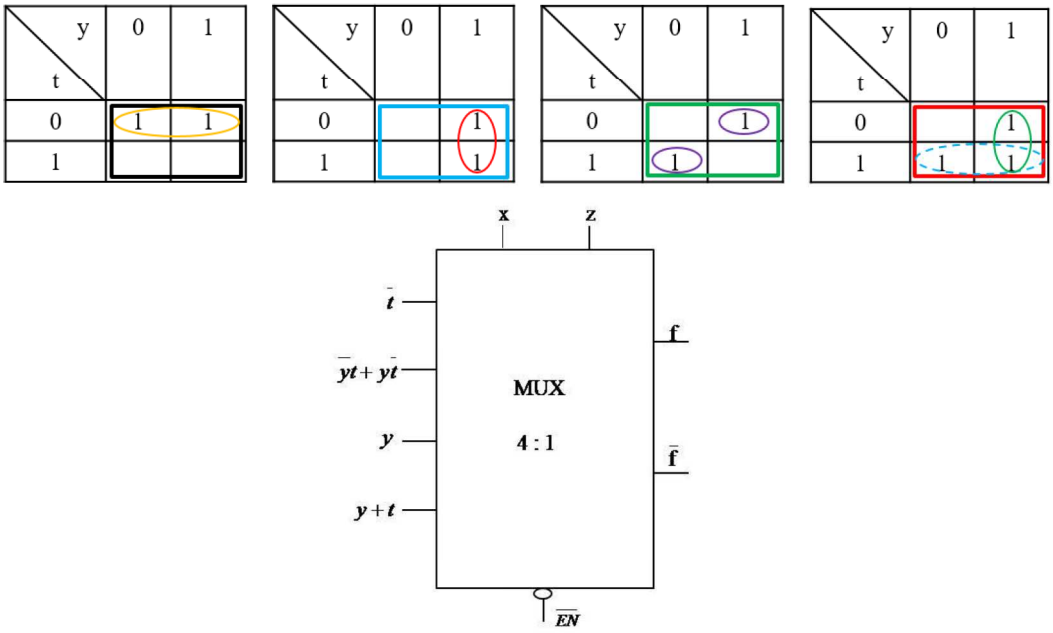
z \ xy	00	01	11	10
0	1	1	1	
1		1	1	

z \ xy	00	01	11	10
0			1	
1	1		1	1

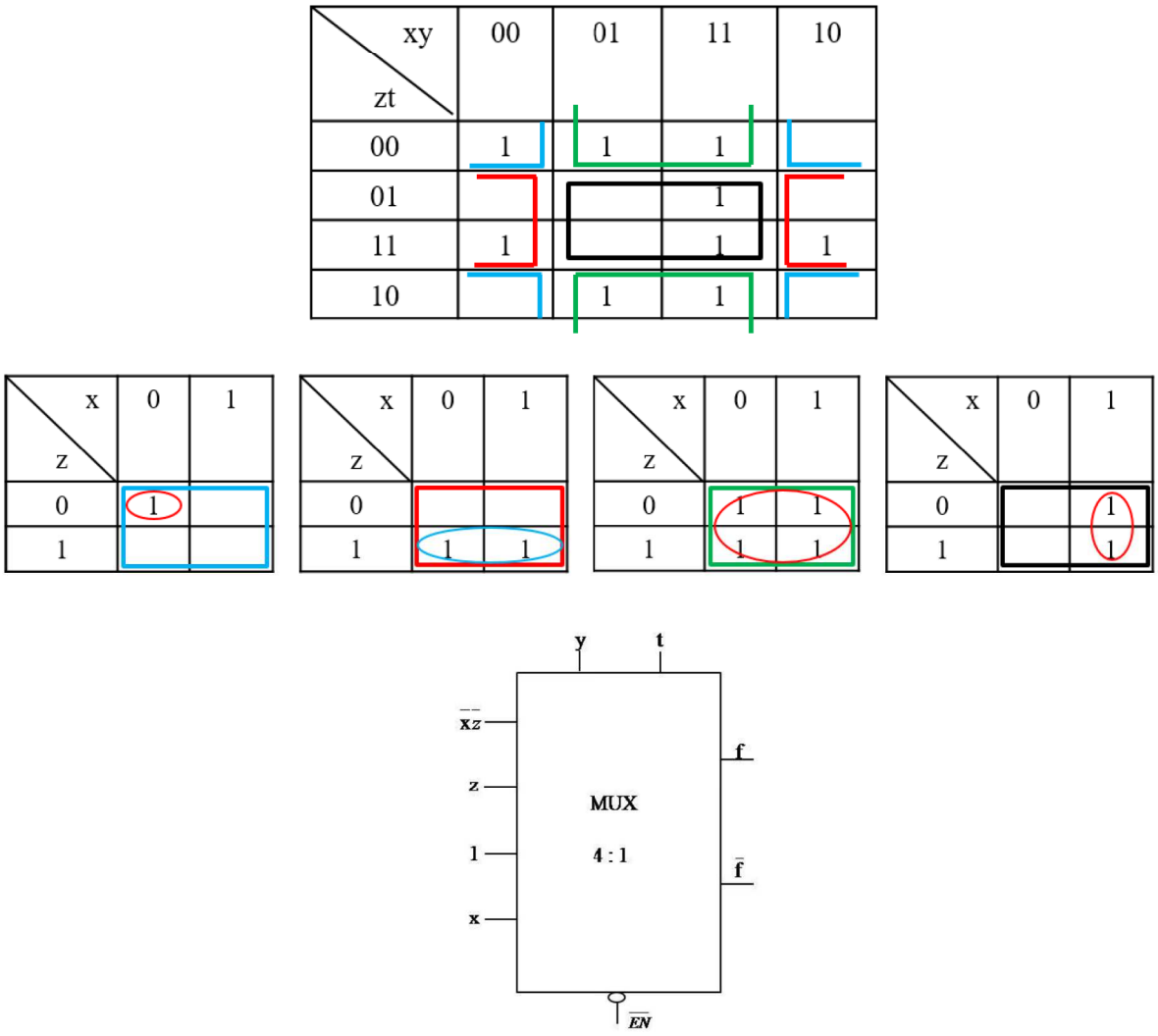


Exemplul 9: Să se implementeze funcția de mai folosind un singur multiplexor 4:1 având ca intrări de selecție x si z:

zt \ xy	00	01	11	10
00	1	1	1	
01			1	
11	1		1	1
10		1	1	



Exemplul 10: Să se implementeze funcția de mai sus folosind un singur multiplexor 4:1 având ca intrări de selecție y si t:



Exemplul 11: Implementați funcția de mai jos utilizând 1 MUX 2:1

x	y	z	t	**f**
0	0	0	0	**A**
0	0	0	1	**1**
0	0	1	0	**B**
0	0	1	1	**0**
0	1	0	0	*
0	1	0	1	**B**
0	1	1	0	**A**
0	1	1	1	**A**
1	0	0	0	**1**
1	0	0	1	**B**
1	0	1	0	**0**
1	0	1	1	*
1	1	0	0	**B**
1	1	0	1	**A**
1	1	1	0	**A**
1	1	1	1	**1**

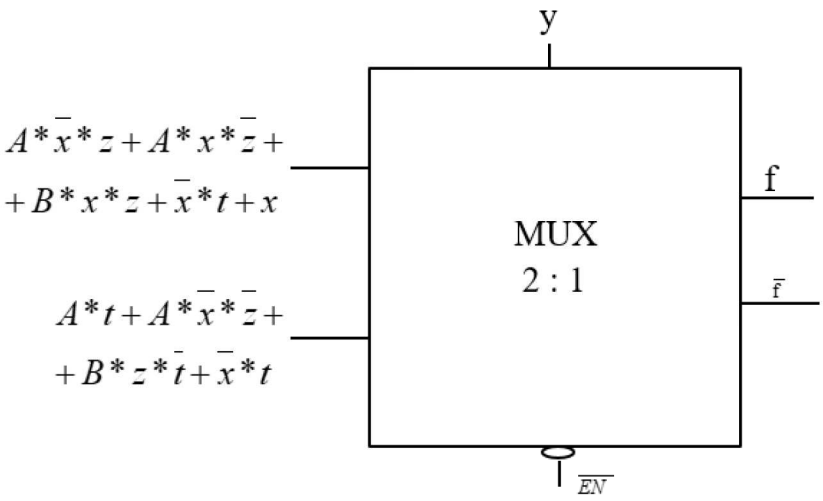
zt \ xy	00	01	11	10
00		A		*
01	*	1	*	A
11	1	*	A	B
10	A	B	B	1

zt \ x	0	1
00		*
01	*	A
11	1	B
10	A	1

zt \ x	0	1
00	A	
01	1	*
11	*	A
10	B	B

$$f_1(x,z,t) = A*\bar{x}*z + A*x*\bar{z} + B*x*z + \bar{x}*t + x*\bar{t}$$

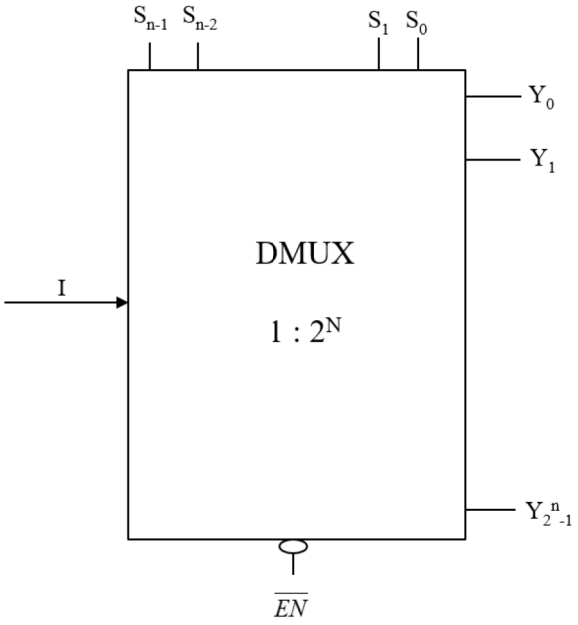
$$f_2(x,z,t) = A*t + A*\bar{x}*\bar{z} + B*z*\bar{t} + \bar{x}*t$$



Demultiplexoare (DMUX)

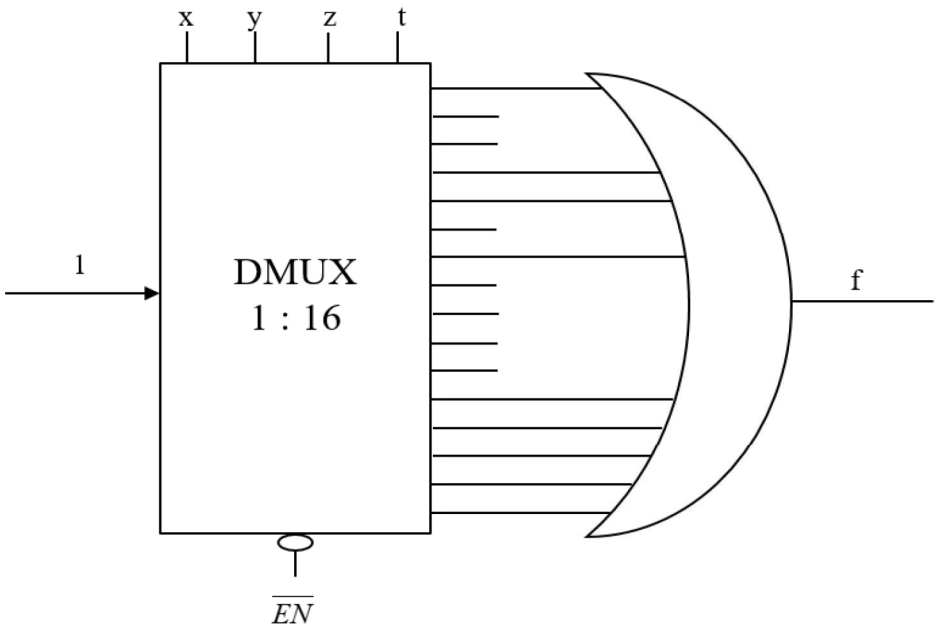
Demultiplexoarele sunt circuite complementare multiplexoarelor care permit transmiterea datelor de pe o intrare de date comuna pe una din ieșirile selectate. Un demultiplexor are o intrare de date, o intrare de enable, n intrări de selecție si 2^n ieșiri.

$Y_k = I;$
$k = (S_{n-1}....S_0)_{10}$



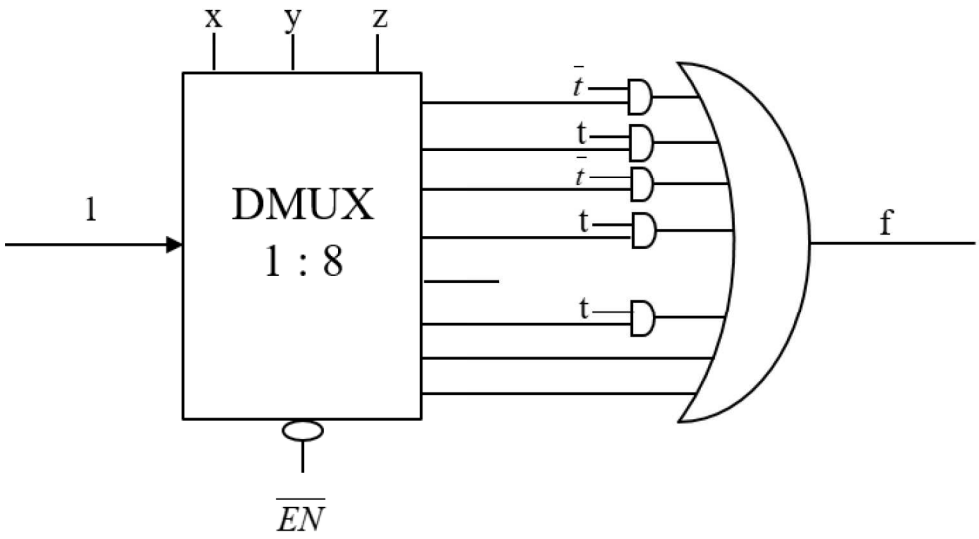
Exemplu 1: Sa se implementeze funcția de mai jos folosind un demultiplexor DMUX 1:16.

x	y	z	t	**f**
0	0	0	0	**1**
0	0	0	1	**0**
0	0	1	0	**0**
0	0	1	1	**1**
0	1	0	0	**1**
0	1	0	1	**0**
0	1	1	0	**1**
0	1	1	1	**0**
1	0	0	0	**0**
1	0	0	1	**0**
1	0	1	0	**0**
1	0	1	1	**1**
1	1	0	0	**1**
1	1	0	1	**1**
1	1	1	0	**1**
1	1	1	1	**1**



Exemplu 2: Sa se implementeze funcția de mai sus folosind un demultiplexor DMUX 1:8.

x	y	z	f
0	0	0	$\bar{t}$
0	0	1	**t**
0	1	0	$\bar{t}$
0	1	1	**t**
1	0	0	**0**
1	0	1	**t**
1	1	0	**1**
1	1	1	**1**



Exerciții propuse:

1. Determinați funcția echivalenta a lui f si construiți tabelul de adevăr al celor doua funcții:

   $f(x,y,z) = x\bar{y}z + \bar{y}\bar{z} + xy + z$

   $g(x,y,z) = x\bar{y}z + x\bar{y}\bar{z} + \bar{x}\bar{y}z + xyz + xy\bar{z} + \bar{x}yz + \bar{x}\bar{y}z$

x	y	z	f	g
0	0	0	**1**	**1**
0	0	1	**1**	**1**
0	1	0	**0**	**0**
0	1	1	**1**	**1**
1	0	0	**1**	**1**
1	0	1	**1**	**1**
1	1	0	**1**	**1**
1	1	1	**1**	**1**

2. Realizați circuitul funcției de mai jos folosind numai porți NAND:
   $f' = x * \overline{y} + \overline{y} * z + \overline{x} * y$
3. Realizați circuitul funcției de mai jos folosind numai porți NAND:
   $f'' = \overline{x} * \overline{y} * \overline{t} + \overline{x} * z * \overline{t} + \overline{x} * y * \overline{z} * t + x * y * z * t + x * \overline{y} * \overline{z}$
4. Realizați circuitul funcției de mai jos folosind numai porți NAND:
   $f'' = \overline{x} * \overline{y} * \overline{t} + \overline{x} * y * \overline{z} * t + x * y * z * \overline{t} + x * \overline{y} * \overline{z} * \overline{t} + x * \overline{y} * \overline{z} * t$
5. Implementați funcția de mai jos folosind un MUX 8:1.

x	y	z	f
0	0	0	**1**
0	0	1	*
0	1	0	**1**
0	1	1	**t**
1	0	0	**0**
1	0	1	**1**
1	1	0	*
1	1	1	**1**

6. Implementați funcția de la exercițiul anterior folosind un DMUX 8:1.
7. Implementați funcția de la exercițiul 1 folosind o structură cu multiplexoare cuplate în cascadă, având pe primul nivel MUX-uri 4:1 comandate de variabilele y, x (în această ordine), iar pe al doilea un MUX 2:1.
8. Implementați funcția de la exercițiul 1 folosind MUX-uri 2:1 cuplate în cascadă.
9. Implementați funcția de mai jos folosind un MUX 16:1.

x	y	z	t	f
0	0	0	0	**0**
0	0	0	1	**1**
0	0	1	0	*
0	0	1	1	**1**
0	1	0	0	*
0	1	0	1	*
0	1	1	0	**1**
0	1	1	1	**0**
1	0	0	0	**0**
1	0	0	1	**1**
1	0	1	0	**1**
1	0	1	1	*
1	1	0	0	**0**
1	1	0	1	**1**
1	1	1	0	**0**
1	1	1	1	*

10. Implementați funcția de la exercițiul anterior folosind un DMUX 16:1.
11. Implementați funcția de la exercițiul 5 folosind MUX-uri 4:1 cuplate în cascadă și

având variabilele y, t pe primul nivel și x, z pe al doilea.

12. Implementați funcția de la exercițiul 5 folosind MUX-uri 4:1 cuplate în cascadă și având variabilele t, x pe primul nivel și y, z pe al doilea.
13. Implementați funcția de la exercițiul 1 folosind un singur MUX 4:1, având variabilele de selecție x și z.